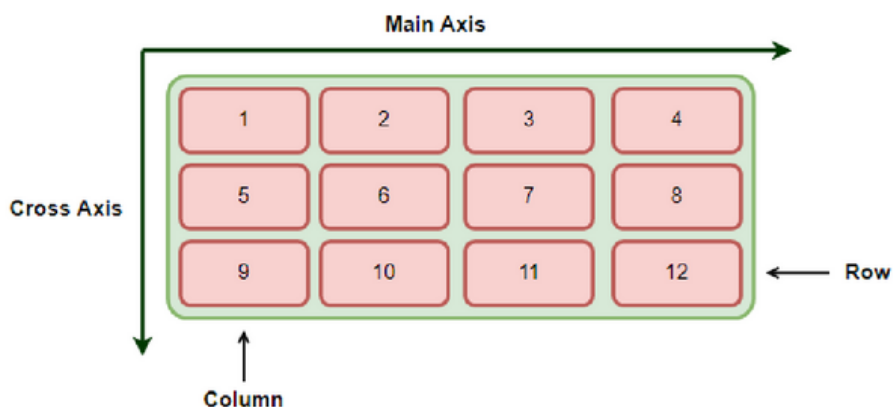


CSS, Grid

CSS Grid

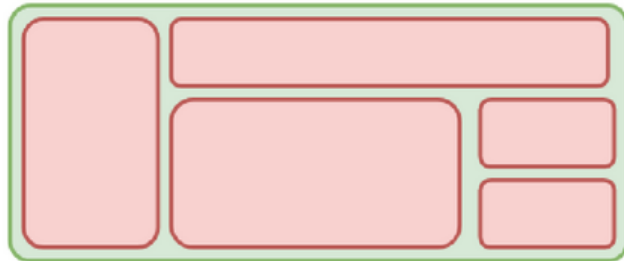
CSS Grid — это модель (шаблон) для сайта, оптимизированный для создания двумерных макетов.

Grid - это набор пересекающихся горизонтальных и вертикальных линий, которые создают размерность и позиционируют систему координат для контента в самом grid-контейнере.



CSS Grid

CSS Grid позволяет объединять строки и столбцы макета:



CSS Grid

Для создания Grid-макета необходимо создать HTML-элемент (**grid-контейнер**) и в его стилях указать:

`display: grid | inline-grid;`

Это автоматически сделает всех прямых потомков этого элемента — **grid-элементами** (дочерними элементами).

Значения свойства:

- **grid** - контейнер, который "снаружи" имеет свойства блочного элемента;
- **inline-grid** - контейнер, который "снаружи" имеет свойства строчного элемента.

Единицы измерения

В свойствах grid для обозначения размеров строк и столбцов можно использовать стандартные единицы измерения (**px**, **em**, % и др.).

Можно еще использовать новую единицу измерения **fr**(fractional unit).

В контексте Grid-макета **fr** можно определить единицей **гибкости**.

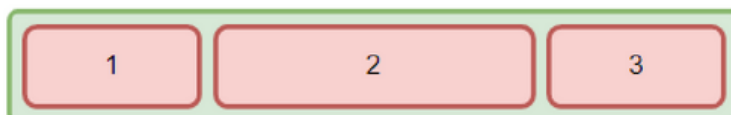
В качестве значений **fr** можно использовать как целые (**2fr**), так и вещественные числа(**1.5fr**).

Единицы измерения

Если указать ширину в **1fr** для каждого дочернего элемента , ширина grid-элемента будет равномерно разделена среди его дочерних элементов:



Если же указать для одного дочернего элемента **2fr**, а для остальных **1fr**, получим следующий результат:



Создание Grid-сетки

Grid-макет представляет собой сетку, количество строк и столбцов которой определяется свойствами:

- **grid-template-rows** - определяет количество строк в grid-контейнере и размер каждого дочернего элемента;
- **grid-template-columns** - определяет количество столбцов в grid-контейнере и размер каждого дочернего элемента.

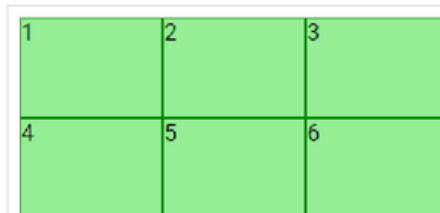
Точное значение размера

Создадим grid-контейнер, состоящий из двух строк и трех столбцов, разместим на сетке 6 одинаковых элементов.

Код CSS

```
.container{  
  display: grid;  
  grid-template-rows: 1fr 1fr;  
  grid-template-columns: 1fr 1fr 1fr;  
  ...  
}
```

Результат



Точное значение размера

Создадим grid-контейнер, состоящий из двух строк и трех столбцов, разместим на сетке 6 одинаковых элементов.

Код CSS

```
.container{  
  display: grid;  
  grid-template-rows: 1fr 3fr;  
  grid-template-columns: 1fr 3fr 2fr;  
  ...  
}
```

Результат

1	2	3
4	5	6

Оператор повторения

Создадим grid-контейнер, состоящий из двух строк и трех столбцов, разместим на сетке 6 одинаковых элементов.

Код CSS

```
.container{  
  display: grid;  
  grid-template-rows: repeat(2, 1fr);  
  grid-template-columns: repeat(3, 1fr);  
  ...  
}
```

Результат

1	2	3
4	5	6

Оператор повторения

Допускается одновременно использовать и оператор повторения, и точное значение размеров элементов сетки.

Код CSS

```
.container{  
  display: grid;  
  grid-template-rows: repeat(2, 1fr);  
  grid-template-columns: 50px repeat(2, 1fr);  
  ...  
}
```

Результат

1	2	3
4	5	6

Размер элементов

При создании grid-сетки можно использовать значения:

- **auto** - заполнение строкой/столбцом всего доступного пространства.

Если для определения ширины/высоты столбцов/строк используются точные значения, то для одного столбца/строки можно указать значение **auto**. Это означает, что соответствующий "захватит" все оставшееся пространство.

Код CSS

```
.container{  
  display: grid;  
  grid-template-columns: 100px auto 20%;  
  ...  
}
```

Результат

grid-container	grid-template-rows	grid-template-columns
----------------	--------------------	-----------------------

Размер элементов

При создании grid-сетки можно использовать значения:

- **max-content** - специальное значение, представляющее собой «идеальный размер» ячейки.

Это наименьший возможный размер ячейки, при котором содержимое в ней уместается "свободно". Например, если содержимое ячейки — предложение текста, идеальной шириной ячейки будет полная длина этого предложения, если записать его в одну строку без переносов:

`grid-template-columns: max-content max-content 1fr;`

grid-container	grid-template-rows	grid-template-columns
----------------	--------------------	-----------------------

Размер элементов

При создании grid-сетки можно использовать значения:

- **min-content** - специальное значение, представляющее собой наименьший возможный размер, при котором в ячейке еще не возникает переполнения.

Столбец занимает такой наименьший размер, при котором текст переносится на следующую строку по знакам препинания.

`grid-template-columns: min-content min-content 1fr;`

grid-container	grid-template-rows	grid-template-columns
----------------	--------------------	-----------------------

Размер элементов

При создании grid-сетки можно использовать значения:

- minmax()** - позволяет описать минимальное и максимальное значение ширины или высоты столбца или строк. В зависимости от размеров окна браузера, значение может быть любым, принадлежащим интервалу от минимального до максимального значения.

В качестве максимального и минимального значения можно использовать размерность: в пикселях (и других фиксированных единицах), в процентах, в гибких единицах, **min-content**, **max-content**, **auto**.

Размер элементов

При создании grid-сетки можно использовать значения:

- minmax()** - позволяет описать минимальное и максимальное значение ширины или высоты столбца или строк. В зависимости от размеров окна браузера, значение может быть любым, принадлежащим интервалу от минимального до максимального значения.

`grid-template-columns: 1fr minmax(max-content, 50%) 1fr;`

grid-container	grid-template-rows	grid-template-columns
----------------	--------------------	-----------------------

grid-container	grid-template-rows	grid-template-columns
----------------	--------------------	-----------------------

Размер, зависящий от окна

- **repeat(auto-fit, размер)** - при создании grid-сетки можно не указывать количество строк, тогда количество столбцов и строк сетки будет вычисляться автоматически в зависимости от размера окна браузера:

`grid-template-columns: repeat(auto-fit, 150px);`

1	2	3	4	5	6
7	8				

1	2	3
4	5	6
7	8	

Размер, зависящий от окна

- **repeat(auto-fit, minmax())** - при автоматическом подборе количества строк и столбцов можно управлять размером ячеек сетки, например, при помощи **minmax()**:

`grid-template-columns: repeat(auto-fit, minmax(300px, 1fr));`

1	2	3
4	5	6
7	8	

1	2
3	4
5	6
7	8

1
2
3
4
5
6
7
8

Размер, зависящий от окна

- `repeat(auto-fit, значение_1 значение_2 ...)` - позволяет повторять группу из нескольких столбцов, подбирая нужное количество строк grid-сетки:

`grid-template-columns: repeat(auto-fit, 100px 20%);`

1	2	3	4	5	6
7	8	9			

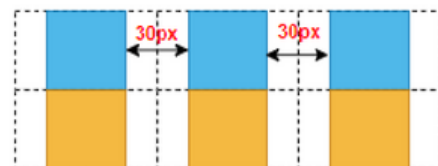
1	2	3	4
5	6	7	8
9			

Интервалы в Grid-сетке

Между столбцами и строками grid-сетки можно добавить интервал, для этого используются свойства:

- для столбцов:

`grid-column-gap: 30px;`



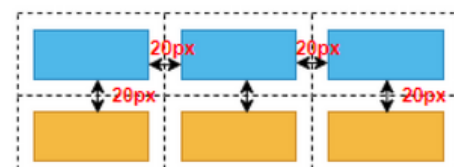
- для строк:

`grid-row-gap: 20px;`



- если интервалы между строками и столбцами совпадают:

`grid-gap: 20px;`

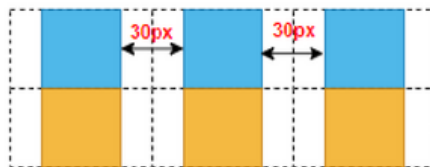


Интервалы в Grid-сетке

Между столбцами и строками grid-сетки можно добавить интервал, для этого используются свойства:

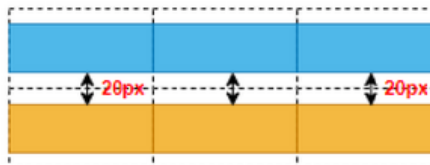
- для столбцов:

`grid-column-gap: 30px;`



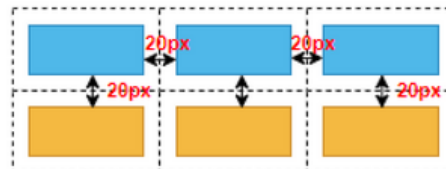
- для строк:

`grid-row-gap: 20px;`



- если интервалы между строками и столбцами совпадают:

`grid-gap: 20px;`

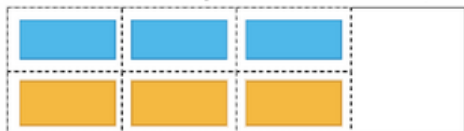


Выравнивание Grid-сетки

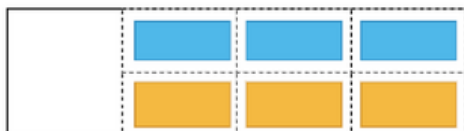
Для выравнивания grid-сетки по главной оси (направлена слева направо) внутри grid-контейнера используется свойство **justify-content**.

Возможные значения свойства:

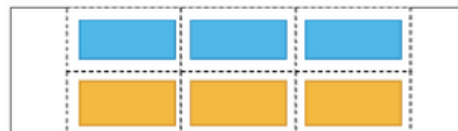
- **start** (по умолчанию)



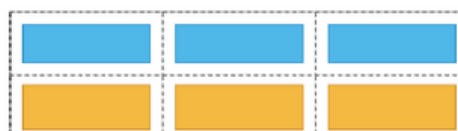
- **end**



- **center**



- **stretch**

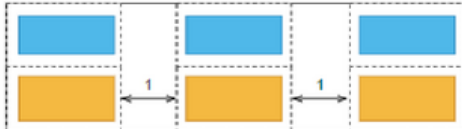


Выравнивание Grid-сетки

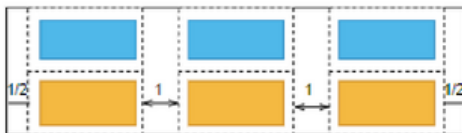
Для выравнивания grid-сетки по главной оси (направлена слева направо) внутри grid-контейнера используется свойство **justify-content**.

Возможные значения свойства:

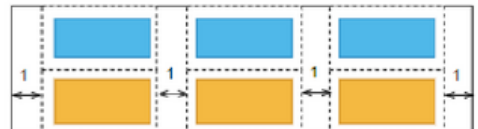
- **space-between**



- **space-around**



- **space-evenly**



Выравнивание элементов

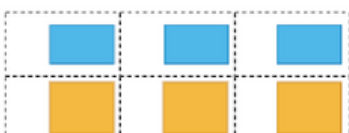
Для выравнивания элементов в grid-сетке по главной оси (направлена слева направо) используется свойство **justify-items**.

Возможные значения свойства:

- **start** (по умолчанию)



- **end**



- **center**



- **stretch**(по умолчанию)

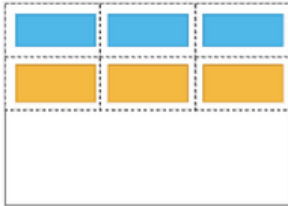


Выравнивание Grid-сетки

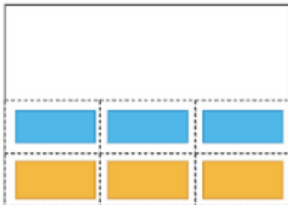
Для выравнивания grid-сетки по поперечной оси (направлена сверху вниз) внутри grid-контейнера используется свойство **align-content**.

Возможные значения свойства:

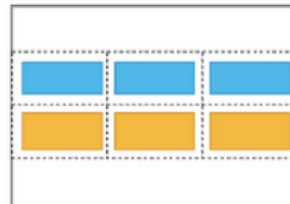
- **start** (по умолчанию)



- **end**



- **center**



- **stretch** (по умолчанию)

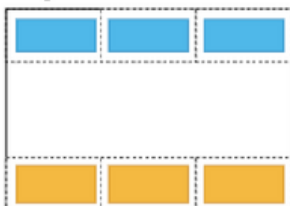


Выравнивание Grid-сетки

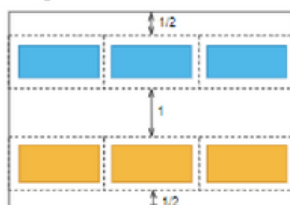
Для выравнивания grid-сетки по поперечной оси (направлена сверху вниз) внутри grid-контейнера используется свойство **align-content**.

Возможные значения свойства:

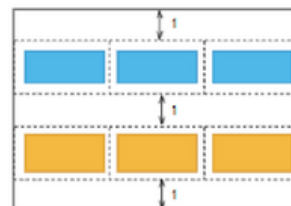
- **space-between**



- **space-around**



- **space-evenly**



Выравнивание элементов

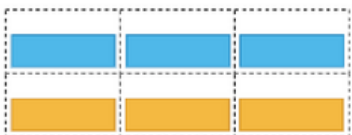
Для выравнивания элементов в grid-сетке по поперечной оси (направлена сверху вниз) используется свойство **align-items**.

Возможные значения свойства:

- **start** (по умолчанию)



- **end**



- **center**



- **stretch**(по умолчанию)



Создание областей Grid-сетки

Ячейки сетки можно объединить (создать **grid-области**), для этого используются свойства:

- для **grid-контейнера**:
 - grid-template-areas**:
 - "шаблон первой строки"
 - "шаблон второй строки"
 - ...
 - "шаблон N-ой строки"
- для дочернего элемента (**grid-элемента**):
 - grid-area**: имя_элемента_шаблона;

Создание областей Grid-сетки

Алгоритм создания областей:

1. Создать сетку страницы, состоящую из необходимого количества строк и столбцов (например. 3 строки и 4 столбца) необходимого размера:



2. Определить объединяемые области:



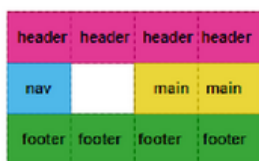
3. Каждой области дать имя:



Создание областей Grid-сетки

4. Описать шаблон всех строки, каждой ячейке строки необходимо сопоставить имя области, при этом:

- если область "захватывает" несколько ячеек строки, то это имя указывается для КАЖДОЙ ячейки;
- если какая-то ячейка (ячейки) пусты, то для них в шаблоне указывается точка.



```
.grid-container {  
  display: grid;  
  grid-template-rows: repeat(3, 50px);  
  grid-template-columns: repeat(4,  
70px);  
  grid-template-areas:  
    "header header header header"  
    "nav . main main"  
    "footer footer footer footer";  
}
```

Создание областей Grid-сетки

header	header	header	header
nav		main	main
footer	footer	footer	footer

5. Для КАЖДОЙ области, определенной в шаблоне:

- создать элемент на HTML-странице;
- создать для него CSS-стиль, в котором указать какой области шаблона этот элемент соответствует.

Код HTML

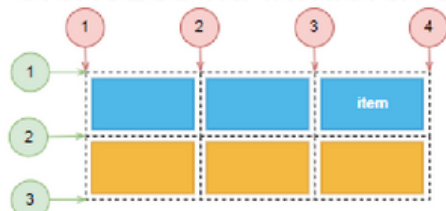
```
<div class="grid-container">
  <div class="block-header">
  </div>
  <div class="block-nav">
  </div>
  <div class="block-main">
  </div>
  <div class="block-footer">
  </div>
</div>
```

Код CSS

```
.grid-container > .block-header {
  grid-area: header;
  background: #D80073;
}
.grid-container > .block-nav {
  grid-area: nav;
  background: #1BA1E2;
}
.grid-container > .block-main {
  grid-area: main;
  background: #E3C800;
}
.grid-container > .block-footer {
  grid-area: footer;
  background: #008A00;
}
```

Расположение элементов

В grid-контейнере линии grid-имеют нумерацию. Нумерация строк и столбцов начинается с 1:



Теперь для каждого grid-элемента можно определить как он расположен по линиям сетки.

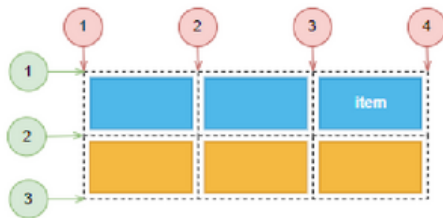
Например, для элемента **item**:

- начальный столбец: 3;
- конечный столбец: 4;
- начальная строка: 1;
- конечная строка: 2.

Расположение элементов

Для расположения grid-элементы по линиям grid-сетки используются свойства:

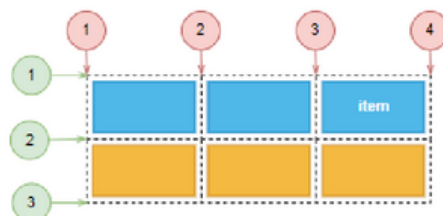
- **grid-column-start** - определяет начальный столбец (3);
- **grid-column-end** - определяет конечный столбец (4);
- **grid-row-start** - определяет начальную строку (1);
- **grid-row-end** - определяет конечную строку (2);
- **grid-column** - определяет начальный и конечный столбец (3 / 4);
- **grid-row** - определяет начальную и конечную строку (1 / 2).



Расположение элементов

Для расположения grid-элементы по линиям grid-сетки используются свойства:

- **grid-column-start** - определяет начальный столбец (3);
- **grid-column-end** - определяет конечный столбец (4);
- **grid-row-start** - определяет начальную строку (1);
- **grid-row-end** - определяет конечную строку (2);
- **grid-column** - определяет начальный и конечный столбец (3 / 4);
- **grid-row** - определяет начальную и конечную строку (1 / 2);
- **grid-area** - определяет положение элементов по строкам и столбцам (1 / 3 / 2 / 4);



Расположение элементов

Алгоритм расположения элементов по линиям сетки

1. Создать сетку страницы, состоящую из необходимого количества столбцов (например, 3) и разместить элементы в сетке:

1	2	3
4	5	6
7	8	

2. Для одного из элементов укажем его положение в сетке (например, блок 2 - 1 / 2 / 3 / 3)

1	2	3
4		5
6	7	8

Расположение элементов

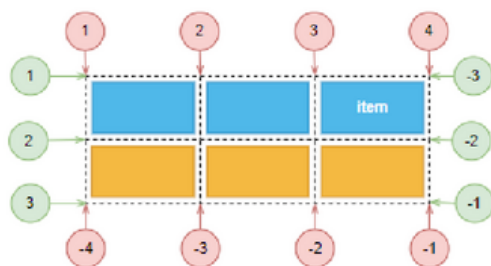
3. Для следующего элементов укажем его положение в сетке (например, блок 6 - 3 / 1 / 4 / 4)

1	2	3
4		5
6		
7	8	

4. Продолжить размещать элементы по сетке.

Расположение элементов

В grid-контейнере линии grid-имеют нумерацию. Нумерация строк и столбцов начинается с 1, но можно использовать нумерацию с нижнего правого угла, тогда нумерация начинается с -1:

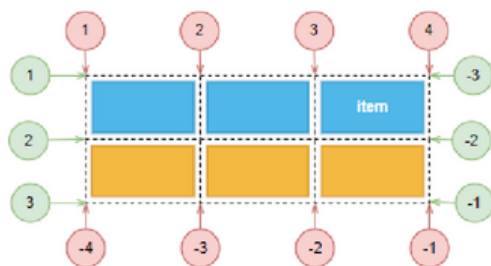


Например, расположение элемента **item** можно описать несколькими спосо

```
.item {  
  grid-column: 3 / 4; /* или 4 / 3, или -1 / -2, или -2 / -1, или 3 / -1, или -1 / 3 */  
  grid-row: 1 / 2; /* или 2 / 1, или -2 / -3, или -3 / -2, или 1 / -2, или -2 / 1 */  
}
```

Расположение элементов

В grid-контейнере линии grid-имеют нумерацию. Нумерация строк и столбцов начинается с 1, но можно использовать нумерацию с нижнего правого угла, тогда нумерация начинается с -1:

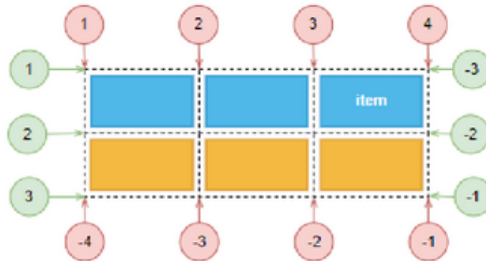


Например, расположение элемента **item** можно описать несколькими спосо

```
.item {  
  grid-column: 3 / 4; /* или 4 / 3, или -1 / -2, или -2 / -1, или 3 / -1, или -1 / 3 */  
  grid-row: 1 / 2; /* или 2 / 1, или -2 / -3, или -3 / -2, или 1 / -2, или -2 / 1 */  
}
```

Расположение элементов

Если элемент занимает одну ячейку, можно не указывать значение через /:

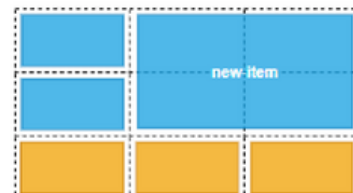


```
.item {  
  grid-column: 3;  
  grid-row: 1;  
}
```

Расположение элементов

С помощью специального слова **span** можно задать растяжение элемента на несколько ячеек. После слова **span** указывается, на какое количество ячеек надо растянуть элемент:

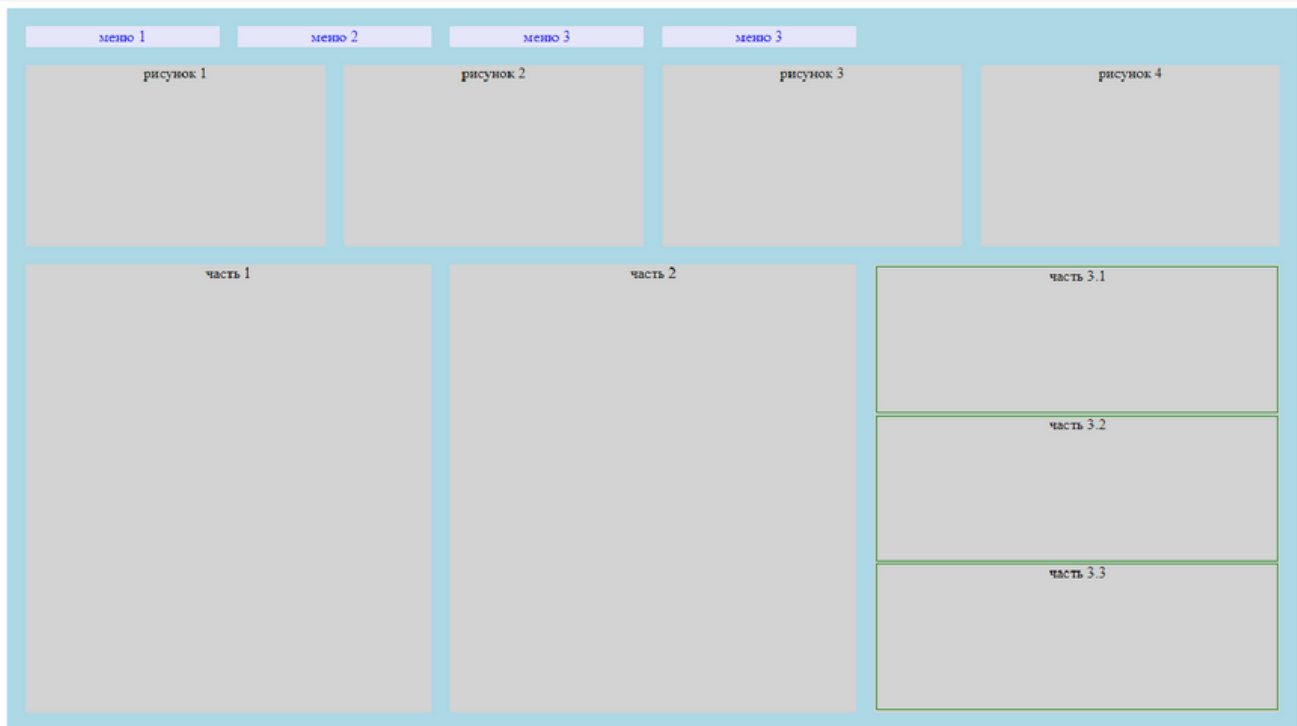
```
.new-item {  
  grid-row: 1 / span 2;  
  grid-column: 2 / span 2;  
}
```



Элемент помещается в ячейку, которая находится на пересечении первой строки и второго столбца, и растягивается на две строки вниз и на два столбца вправо.

Grid, пример верстки

Пример. С помощью **Grid** создать страницу следующей структуры:



Grid, пример верстки

Шаг 1. Реализуем HTML-код страницы

```
<div class="content menu">
  <a href="#">меню 1</a>
  <a href="#">меню 2</a>
  <a href="#">меню 3</a>
  <a href="#">меню 3</a>
</div>
<div class="content">
  <div class="image">рисунок 1</div>
  <div class="image">рисунок 2</div>
  <div class="image">рисунок 3</div>
  <div class="image">рисунок 4</div>
</div>
<div class="content main">
  <div class="article">часть 1</div>
  <div class="article">часть 2</div>
  <div class="article">
    <div class="item">часть 3.1</div>
    <div class="item">часть 3.2</div>
    <div class="item">часть 3.3</div>
  </div>
</div>
```

Grid, пример верстки

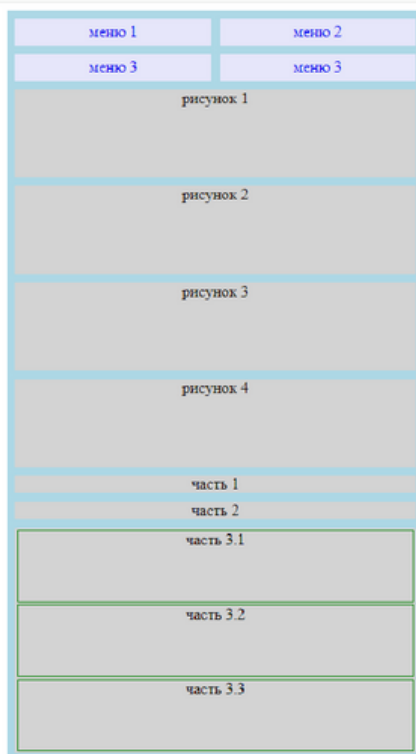
Шаг 2. Описываем стили для каждого класса (без перестроения).

```
.content {
  width: 70%;
  display: grid;
  margin: 0 auto;
  grid-template-columns: repeat(4, 1fr);
  padding: 1vmax 1vmax 0;
  gap: 1vmax;
  text-align: center;
  background: lightblue;
}
.main {
  grid-template-columns: repeat(3, 1fr);
  padding: 1vmax;
}
.menu{
  grid-template-columns: repeat(6, 1fr);
}
```

```
.article {
  background: lightgrey;
}
.image{
  height: 10vmax;
  background: lightgrey;
}
.item {
  margin: 0.5%;
  height: 8vmax;
  border: thin solid green;
}
a {
  padding: 0.5vmax;
  text-decoration: none;
  background: Lavender;
}
```

Grid, пример верстки

При уменьшении ширины окна браузера элементы должны перестраиваться (в элементах часть 1 и часть 2 нет содержимого):



Grid, пример верстки

Шаг 3. Добавляем медиа запрос, в котором изменяем количество колонок в grid сетках.

```
@media (max-width:900px){  
  .content{  
    grid-template-columns: 1fr;  
  }  
  .menu{  
    grid-template-columns: 1fr 1fr;  
  }  
  .main{  
    grid-template-columns: 1fr;  
  }  
}
```

Спасибо за внимание!